

Systeme de Gestion de Bibliotheque Universitaire

Janvier 2026

Table des matières

1. INTRODUCTION	2
1.1 Contexte	2
1.2 Problématique	2
1.3 Objectifs du projet	2
1.4 Méthodologie	3
2. ANALYSE ET MODÉLISATION	3
2.1 Identification des entités principales	3
2.2 Diagramme de classes UML (Simplifié)	5
2.3 Règles métier implémentées	5
3. CONCEPTION TECHNIQUE	6
3.1 Architecture du système	6
3.2 Description détaillée des classes	7
3.3 Patterns de conception utilisés.....	12
3.4 Gestion de la persistance.....	13
4. IMPLÉMENTATION.....	15
4.1 Technologies utilisées.....	15
4.2 Structure du projet.....	15
4.3 Fonctionnalités implémentées.....	15
4.4 Interfaces utilisateur.....	16
5. TESTS ET VALIDATION.....	17
5.1 Méthodologie de test.....	17
5.2 Scénarios testés.....	17
6. DIFFICULTÉS RENCONTRÉES ET SOLUTIONS.....	18
6.1 Gestion des dépendances circulaires.....	18
6.2 Persistance de l'historique des emprunts.....	18
6.3 Gestion des compteurs globaux.....	18
6.4 Encodage des caractères spéciaux.....	18
6.5 Validation des saisies utilisateur.....	19
6.6 Synchronisation interface-métier.....	19
7. AMÉLIORATIONS POSSIBLES.....	19
7.1 Court terme.....	19
7.2 Moyen terme.....	19
7.3 Long terme.....	19

8. CONCLUSION	19
8.1 Objectifs atteints	20
8.2 Compétences développées.....	20
8.3 Réflexions personnelles.....	20
8.4 Remerciements.....	20
9. BIBLIOGRAPHIE.....	21
10. ANNEXES	21
Annexe A : Extraits du fichier JSON.....	21
Annexe B : Extrait du fichier de logs.....	22
Annexe C : Extrait notifications.....	22
Annexe D : Exemple de statistiques CSV	22

Dakar Institute of Technology (DIT)
Master 1 Intelligence Artificielle

Projet Python - Examen Pratique

Présenté par : SY MAMADOU , SAMBA Japhet Arpaxad , KOYAKPEGON Josué , TONI Lowobien Job Régis

Encadré par : M. Madicke Diop

Janvier 2026

1. INTRODUCTION

1.1 Contexte

Le Dakar Institute of Technology (DIT) dispose d'une bibliothèque académique destinée aux étudiants, enseignants et au personnel administratif. Jusqu'à présent, la gestion de cette bibliothèque était effectuée de manière entièrement manuelle : enregistrement des livres sur cahier, suivi approximatif des emprunts, absence d'historique fiable et difficultés à produire des statistiques exploitables.

Cette situation entraînait plusieurs problèmes majeurs : - Pertes de données fréquentes - Impossibilité de suivre l'historique des emprunts - Difficultés à identifier les livres populaires - Gestion inefficace des réservations - Absence de système de pénalités pour les retards

1.2 Problématique

Comment digitaliser efficacement la gestion d'une bibliothèque universitaire tout en garantissant : - La traçabilité de toutes les opérations - La sécurité et la persistance des données - L'évolutivité du système - Une interface utilisateur intuitive - Le respect des règles métier spécifiques

1.3 Objectifs du projet

Les objectifs principaux de ce projet sont :

1. **Automatiser la gestion du catalogue** : Permettre l'ajout, la modification et la suppression de livres et d'exemplaires
2. **Gérer les utilisateurs** : Inscrire différents types d'utilisateurs avec des privilèges adaptés
3. **Faciliter les transactions** : Simplifier les emprunts, retours et renouvellements
4. **Implémenter un système de réservation** : Gérer une file d'attente avec notifications
5. **Fournir des statistiques exploitables** : Permettre l'analyse des données avec export CSV
6. **Assurer la persistance** : Sauvegarder automatiquement toutes les données
7. **Garantir l'audit** : Tracer toutes les opérations dans un fichier de log

1.4 Méthodologie

Le développement a suivi une approche **orientée objet** stricte en Python, avec : - Architecture modulaire respectant le principe de responsabilité unique - Encapsulation complète des attributs avec propriétés Python - Utilisation de l'héritage pour les types d'utilisateurs - Polymorphisme via une classe abstraite (ABC) - Persistance via sérialisation/désérialisation JSON - Documentation complète du code - Tests fonctionnels de chaque fonctionnalité

2. ANALYSE ET MODÉLISATION

2.1 Identification des entités principales

L'analyse du domaine a permis d'identifier 7 entités principales :

2.1.1 Livre

Représente une référence bibliographique unique identifiée par son ISBN. Un livre peut avoir plusieurs exemplaires physiques.

Attributs clés : - ISBN (identifiant unique) - Titre, Auteur, Catégorie - Année de publication - Statut (via Enum : DISPONIBLE, EMPRUNTE, RESERVE, PERDU, ENDOMMAGE) - Compteur d'emprunts total (statistique)

2.1.2 Exemple

Représente une copie physique d'un livre. C'est l'exemplaire qui est emprunté, pas le livre lui-même.

Attributs clés : - ID unique (ex : EX001-1) - Référence au livre - Statut propre à cet exemplaire

2.1.3 Utilisateur (Classe abstraite)

Classe de base abstraite représentant un membre de la bibliothèque.

Attributs communs : - ID généré automatiquement avec préfixe (ETU-, ENS-, ADM-)
- Nom, Email - Type d'utilisateur - Liste des emprunts en cours - Historique complet des emprunts - Limite d'emprunt maximale (variable selon le type) - Compteur d'emprunts total

Sous-classes : - **Etudiant** : Limite de 3 livres, attributs spécifiques (niveau_etude, filiere) - **Enseignant** : Limite de 5 livres, attribut département - **PersonnelAdministratif** : Limite de 7 livres, attribut service

2.1.4 Emprunt

Représente une transaction de prêt d'un exemplaire à un utilisateur.

Attributs clés : - ID unique (ex : EMP-1) - Exemplaire emprunté - Utilisateur emprunteur
- Date d'emprunt - Date de retour prévue - Date de retour effective - Statut (actif/clôturé)

Méthodes importantes : - `est_en_retard()` : Vérifie si le retour est en retard - `calculer_penalite()` : Calcule la pénalité (100 unités/jour) - `renouveler_emprunt()` : Prolonge de 7 jours

2.1.5 Reservation

Gère la file d'attente pour un livre indisponible.

Attributs clés : - ID unique (ex : RES-1) - Livre réservé - Utilisateur - Date de demande

Méthode importante : - `notifier_disponibilite()` : Écrit dans le fichier de notifications

2.1.6 Bibliotheque

Classe centrale orchestrant toutes les opérations du système.

Collections gérées : - Catalogue (dict[isbn, Livre]) - Exemplaires (dict[id_exemplaire, Exemplaire]) - Utilisateurs (dict[id_utilisateur, Utilisateur]) - Emprunts en cours (dict[id_emprunt, Emprunt]) - Réservations (dict[isbn, list[Reservation]])

2.2 Diagramme de classes UML (Simplifié)

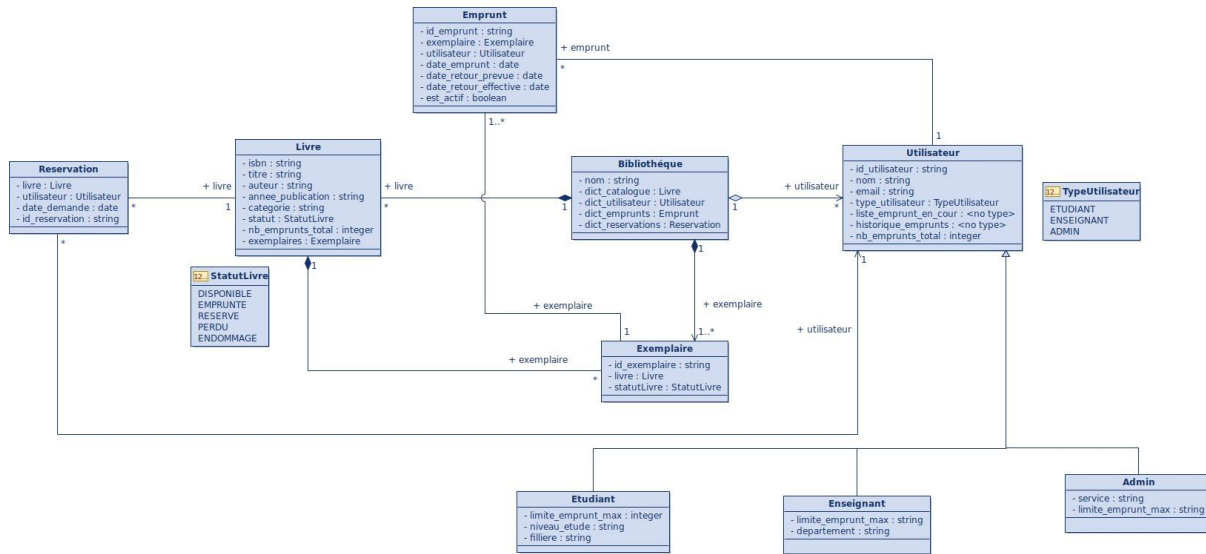


FIG. 1 : Diagramme de classes

Ce diagramme présente les relations entre les entités principales du système.

2.3 Règles métier implémentées

Limites d'emprunt par type d'utilisateur

- **Étudiant** : 3 livres maximum simultanément
- **Enseignant** : 5 livres maximum simultanément
- **Personnel Administratif** : 7 livres maximum simultanément

Durées et délais

- **Durée d'emprunt standard** : 14 jours
- **Durée de renouvellement** : +7 jours
- **Condition de renouvellement** : Aucune réservation en attente sur ce livre

Pénalités

- **Montant** : 100 unités monétaires par jour de retard
- **Calcul** : $(date_actuelle - date_retour_prevue) \times 100$
- **Blocage** : Un utilisateur avec des livres en retard ne peut pas emprunter

Réservations

- **File d'attente FIFO** : Premier arrivé, premier servi
- **Notification automatique** : Lorsqu'un exemplaire devient disponible

- **Statut réservé** : L'exemplaire passe en statut RESERVE (non DISPONIBLE)

Identifiants uniques

- **ISBN** : Identifie un livre de manière unique dans le catalogue
 - **ID Exemplaire** : Format EX001-N (compteur auto-incrémenté)
 - **ID Utilisateur** : Format PREFIXE-N (ETU-1, ENS-2, ADM-3)
 - **ID Emprunt** : Format EMP-N
 - **ID Réservation** : Format RES-N
-

3. CONCEPTION TECHNIQUE

3.1 Architecture du système

Le projet suit une architecture modulaire en couches :

Système de Gestion de Bibliothèque/

Couche Présentation (Interfaces)

main.py	# Interface CLI standard
main_interactive.py	# Interface interactive (InquirerPy)

Couche Métier (Logique)

bibliotheque.py	# Orchestrateur central
-----------------	-------------------------

Couche Domaine (Entités)

livre.py	# Entité Livre + Enum StatutLivre
exemplaire.py	# Entité Exemplaire
utilisateur.py	# Classe abstraite Utilisateur
etudiant.py	# Sous-classe Etudiant
enseignant.py	# Sous-classe Enseignant
admin.py	# Sous-classe PersonnelAdministratif
emprunt.py	# Entité Emprunt
reservation.py	# Entité Reservation

Couche Persistance

data_bibliotheque.json	# Base de données JSON
bibliotheque.log	# Journal d'audit
notifications_reservations.txt	# Notifications

Avantages de cette architecture : - **Séparation des préoccupations** : Chaque couche a une responsabilité claire - **Maintenabilité** : Modifications localisées sans impact global - **Testabilité** : Chaque composant peut être testé indépendamment - **Évolutivité** : Possibilité d'ajouter facilement de nouvelles interfaces ou fonctionnalités

3.2 Description détaillée des classes

3.2.1 Classe Livre

Fichier : livre.py

Responsabilité : Représenter une référence bibliographique unique

Attributs privés :

- __isbn: str *# Identifiant unique (clé primaire)*
- __titre: str *# Titre du livre*
- __auteur: str *# Auteur principal*
- __anne_publication: int *# Année de publication*
- __categorie: str *# Catégorie (Roman, Science-Fiction, etc.)*
- __statut: StatutLivre *# Statut global (Enum)*
- __nb_emprunts_total: int *# Compteur pour statistiques*

Propriétés (Getters/Setters) : - Tous les attributs disposent de getters - Setters pour : titre, auteur, categorie, anne_publication, statut - ISBN en lecture seule (immutable après création)

Méthodes principales :

- + incrementer_compteur() *# Incrémente nb_emprunts_total*
- + to_dict() -> dict *# Sérialisation pour JSON*
- + from_dict(data) -> Livre *# Désérialisation (class method)*
- + __str__() -> str *# Représentation textuelle*

Énumération StatutLivre :

```
class StatutLivre(Enum):  
    DISPONIBLE = "disponible"  
    EMPRUNTE = "emprunté"  
    RESERVE = "réservé"  
    PERDU = "perdu"  
    ENDOMMAGE = "endommagé"
```

Exemple d'utilisation :

```
livre = Livre(  
    isbn="978-0123456789",  
    titre="Le Petit Prince",  
    auteur="Antoine de Saint-Exupéry",  
    anne_publication=1943,  
    categorie="Jeunesse"  
)  
livre.incrementer_compteur() # Pour statistiques
```

3.2.2 Classe Exemple

Fichier : exemple.py

Responsabilité : Représenter une copie physique d'un livre

Attributs privés :

- __id_exemple: str *# ID unique (ex: EX001-1)*
- __livre: Livre *# Référence au livre parent*
- __statut: StatutLivre *# Statut propre à cet exemple*

Attribut de classe :

compteur: int = 1 *# Compteur global pour génération d'ID*

Méthodes principales :

- + to_dict() -> dict *# Sauvegarde isbn_livre et statut*
- + from_dict(data, catalogue) -> Exemple *# Reconstruction*

Logique importante : - Un exemple est toujours lié à un livre du catalogue - Le statut de l'exemple est indépendant du statut du livre - L'ID est généré automatiquement et incrémenté globalement

3.2.3 Classe Utilisateur (ABC)

Fichier : utilisateur.py

Responsabilité : Classe abstraite définissant le contrat pour tous les utilisateurs

Attributs privés :

- __id_utilisateur: str *# ID généré avec préfixe*
- __nom: str
- __email: str
- __type_utilisateur: str *# "ETUDIANT", "ENSEIGNANT", "ADMIN"*
- __liste_emprunt_en_cour: list[Emprunt]
- __historique_emprunts: list[Emprunt]
- __nb_emprunts_total: int *# Compteur statistique*

Attribut protégé :

_limite_emprunt_max: int *# Défini dans les sous-classes*

Méthodes principales :

- + peut_emprunter_livre() -> bool *# Vérifie le quota*
- + incrementer_nb_emprunts() *# Incrmente le compteur*
- + ajouter_a_historique(emprunt) *# Ajoute à l'historique*
- + to_dict() -> dict *# Méthode abstraite*
- + from_dict(data) -> Utilisateur *# Méthode abstraite*

Pattern Template Method : La classe mère définit la structure, les sous-classes implémentent les détails.

3.2.4 Sous-classes d'Utilisateur

Etudiant (etudiant.py) :

Attributs spécifiques:

- __niveau_etude: str *# "L1", "L2", "L3", "M1", "M2"*
- __filiere: str *# "IA", "Génie Logiciel", etc.*
- _limite_emprunt_max = 3

Enseignant (enseignant.py) :

Attributs spécifiques:

- __departement: str *# "Informatique", "Mathématiques", etc.*
- _limite_emprunt_max = 5

PersonnelAdministratif (admin.py) :

Attributs spécifiques:

- __service: str *# "Direction", "Scolarité", etc.*
- _limite_emprunt_max = 7

Chaque sous-classe implémente :

- + to_dict() -> dict *# Inclut les attributs spécifiques*
- + from_dict(data) -> [Type] *# Reconstruction spécifique*

3.2.5 Classe Emprunt

Fichier : emprunt.py

Responsabilité : Gérer le cycle de vie d'un prêt

Attributs privés :

- __id_emprunt: str *# ID unique (EMP-N)*
- __exemplaire: Exempleire *# Exempleire emprunté*
- __utilisateur: Utilisateur *# Emprunteur*
- __date_emprunt: date
- __date_retour_prevue: date
- __date_retour_effective: date *# None si en cours*
- __est_actif: bool

Attributs de classe :

- compteur: int = 1
- penalité: int = 100 *# Montant par jour de retard*
- duree_renouvellement: int = 7 *# Jours ajoutés au renouvellement*

Méthodes principales :

- + est_en_retard() -> bool
 # Retourne True si date_retour_prevue dépassée
- + _calculer_penalité() -> int
 # Calcule : (jours_retard × pénalité)
- + renouveler_emprunt()
 # Ajoute durée_renouvellement à date_retour_prevue
- + retourner_emprunt()
 # Fixe date_retour_effective et est_actif=False
- + to_dict() -> dict
- + from_dict(data, exemplaires, users) -> Emprunt

Logique de calcul de pénalité :

```
def _calculer_penalité(self):
    if self.est_en_retard():
        date_fin = self.date_retour_effective or date.today()
        jours_retard = (date_fin - self.date_retour_prevue).days
        return Emprunt.pénalité * jours_retard
    return 0
```

3.2.6 Classe Reservation

Fichier : reservation.py

Responsabilité : Gérer la file d'attente pour un livre indisponible

Attributs privés :

- __id_reservation: str *# ID unique (RES-N)*
- __livre: Livre *# Livre réservé*
- __utilisateur: Utilisateur *# Utilisateur en attente*
- __date_demande: date *# Date de la réservation*

Méthodes principales :

- + notifier_disponibilite()
 # Écrit dans notifications_reservations.txt
- + to_dict() -> dict
- + from_dict(data, users, livres) -> Reservation

Format de notification :

```
--- NOTIFICATION DE DISPONIBILITÉ ---
Date : 2026-01-06
Utilisateur : Jean Etudiant (ID: ETU-1)
```

Livre disponible : Le Petit Prince (ISBN: 978-0123456789)
Action : Merci de passer à la bibliothèque pour récupérer votre exemplaire.

3.2.7 Classe Bibliotheque

Fichier : bibliotheque.py

Responsabilité : Orchestrateur central gérant toutes les opérations

Attributs privés (Collections) :

- __nom: str
- __catalogue: dict[str, Livre]
- __exemplaires: dict[str, Exemplaire]
- __utilisateurs: dict[str, Utilisateur]
- __emprunt_en_cour: dict[str, Emprunt]
- __reservations: dict[str, list[Reservation]]

Méthodes principales :

Gestion du catalogue :

- + ajouter_au_catalogue(livre: Livre) -> str
- + ajouter_exemplaire(isbn: str) -> str
- + modifier_livre(isbn: str, **kwargs) -> str
- + supprimer_livre(isbn: str) -> str
- + rechercher_livre(isbn: str) -> Livre
- + recherche_par_auteur(auteur: str) -> list[Livre]
- + recherche_par_titre(titre: str) -> list[Livre]
- + recherche_par_categorie(categorie: str) -> list[Livre]
- + recherche_par_mot_clé(mot: str) -> list[Livre]
- + recherche_par_annee(annee: int) -> list[Livre]
- + recherche_par_disponibilite(statut: str) -> list[Exemplaire]

Gestion des utilisateurs :

- + ajouter_utilisateur(user: Utilisateur)
- + modifier_utilisateur(id: str, **kwargs) -> str
- + supprimer_utilisateur(id: str) -> str
- + rechercher_utilisateur(id: str) -> Utilisateur
- + recherche_utilisateurs_par_mot_clé(mot: str) -> list[Utilisateur]

Gestion des emprunts :

- + effectuer_emprunt(id_ex: str, id_user: str) -> str
- + retourner_livre(id_emprunt: str)
- + renouveler_livre(id_emprunt: str) -> str
- + effectuer_reservation(isbn: str, id_user: str) -> str

Statistiques :

- + generer_rapport_statistique() -> dict
- + exporter_statistiques_csv(filename: str) -> str

Persistence :

- + sauvegarder_donnees()
- + charger_donnees()
- + enregistrer_action(action: str, details: str)

3.3 Patterns de conception utilisés

3.3.1 Template Method Pattern

Localisation : Classe Utilisateur (ABC)

La classe mère définit la structure commune (attributs, méthodes), les sous-classes implémentent les spécificités (to_dict, from_dict avec attributs propres).

3.3.2 Factory Pattern

Localisation : Méthodes from_dict() de chaque classe

Ces méthodes de classe agissent comme des factories pour reconstruire des objets depuis des données sérialisées.

```
@classmethod
def from_dict(cls, data):
    # Reconstruction de l'objet avec validation
    return cls(...)
```

3.3.3 Singleton Pattern (implicite)

Localisation : Classe Bibliotheque

Dans l'utilisation pratique, une seule instance de Bibliotheque est créée et partagée dans toute l'application.

3.3.4 Strategy Pattern

Localisation : Limites d'emprunt par type d'utilisateur

Chaque sous-classe d'Utilisateur définit sa propre stratégie de limite (_limite_emprunt_max).

3.3.5 Observer Pattern (simplifié)

Localisation : Système de notifications

Lorsqu'un livre devient disponible, les utilisateurs en attente sont "notifiés" via un fichier texte.

3.4 Gestion de la persistance

3.4.1 Sauvegarde (méthode sauvegarder donnees)

Processus : 1. Création d'un dictionnaire global structuré 2. Sérialisation de toutes les collections via `to_dict()` 3. Écriture dans `data_bibliotheque.json` (format UTF-8, indenté) 4. Déclenchement automatique après chaque modification

Structure JSON :

```
{
  "catalogue": {
    "ISBN1": {...},
    "ISBN2": {...}
  },
  "utilisateurs": {
    "ETU-1": {...},
    "ENS-2": {...}
  },
  "exemplaires": {
    "EX001-1": {...}
  },
  "emprunts": {
    "EMP-1": {...}
  },
  "reservations": {
    "ISBN1": [
      {...},
      {...}
    ]
  }
}
```

Code simplifié :

```
def sauvegarder_donnees(self):
    donnees = {
        "catalogue": {isbn: livre.to_dict() for isbn, livre in self.__catalogue.items()}
        "utilisateurs": {uid: user.to_dict() for uid, user in self.__utilisateurs.items()}
        "exemplaires": {eid: ex.to_dict() for eid, ex in self.__exemplaires.items()}
        "emprunts": {eid: emp.to_dict() for eid, emp in self.__emprunt_en_cour.items()}
        "reservations": {isbn: [r.to_dict() for r in liste] for isbn, liste in self
    }

    with open("data_bibliotheque.json", "w", encoding="utf-8") as f:
        json.dump(donnees, f, indent=4, ensure_ascii=False)
```

3.4.2 Chargement (méthode charger donnees)

Processus : 1. Vérification de l'existence du fichier JSON 2. Lecture et parsing du JSON 3. Reconstruction des objets dans l'ordre de dépendance : - Catalogue (livres) → Pas de dépendance - Utilisateurs → Pas de dépendance - Exemplaires → Dépendent du catalogue - Emprunts → Dépendent des exemplaires et utilisateurs - Réservations → Dépendent des livres et utilisateurs 4. Mise à jour des compteurs globaux pour éviter les doublons d'ID

Gestion des compteurs :

Après chargement des utilisateurs

```
if self.__utilisateurs:
    ids = [int(uid.split("-")[1]) for uid in self.__utilisateurs.keys()]
    Utilisateur.compteur = max(ids) + 1
```

Reconstruction avec typage :

Exemple pour les utilisateurs

```
for uid, data in donnees["utilisateurs"].items():
    u_type = data["type_utilisateur"]
    if u_type == "ETUDIANT":
        self.__utilisateurs[uid] = Etudiant.from_dict(data)
    elif u_type == "ENSEIGNANT":
        self.__utilisateurs[uid] = Enseignant.from_dict(data)
    elif u_type == "ADMIN":
        self.__utilisateurs[uid] = PersonnelAdministratif.from_dict(data)
```

3.4.3 Journalisation (méthode enregistrer action)

Format des logs :

[YYYY-MM-DD HH:MM:SS] - ACTION : Détails

Exemples :

[2026-01-06 15:38:27] - RESERVATION : L'utilisateur ADM-1 a réservé le livre ISBN: 1123

[2026-01-06 15:38:40] - EMPRUNT : L'utilisateur ADM-1 a pris l'exemplaire EX001-4

- `[2026-01-06 15:41:52] - RETOUR : Retour de l'emprunt ID: EMP-1

[2026-01-06 15 :41 :52] - RETOUR : Retour de l'emprunt ID : EMP-3

****Implémentation** :**

```python

```
def enregistrer_action(self, action, details):
 horodatage = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
 ligne_log = f"[{horodatage}] - {action.upper()} : {details}\n"
 with open("bibliotheque.log", "a", encoding="utf-8") as f:
 f.write(ligne_log)
```

**Actions enregistrées** : Catalogue, Exemplaire, Utilisateur, Emprunt, Retour, Réservation, Renouvellement, Modifications, Suppressions.



---

## 4. IMPLÉMENTATION

### 4.1 Technologies utilisées

**Langage** : Python 3.8+

**Bibliothèques standard** : json, datetime, os, csv, re, sys, abc, enum

**Bibliothèque tierce** : InquirerPy (interface interactive)

### 4.2 Structure du projet

|                                      |                           |
|--------------------------------------|---------------------------|
| Bibliothèque/                        |                           |
| main.py                              | # Interface CLI standard  |
| main_interactive.py                  | # Interface interactive   |
| bibliotheque.py                      | # Logique métier centrale |
| livre.py, exemplaire.py              | # Entités du catalogue    |
| utilisateur.py                       | # Classe abstraite        |
| etudiant.py, enseignant.py, admin.py | # Sous-classes            |
| emprunt.py, reservation.py           | # Transactions            |
| data_bibliotheque.json               | # Persistance             |
| bibliotheque.log                     | # Journalisation          |
| README.md                            | # Documentation           |

**Total** : ~3000 lignes de code Python

### 4.3 Fonctionnalités implémentées

#### 4.3.1 Gestion des livres

- **Ajout au catalogue** : Vérification unicité ISBN, sauvegarde automatique
- **Ajout d'exemplaires** : Génération ID automatique (EX001-N)
- **Modification** : Mise à jour dynamique avec kwargs
- **Suppression** : Protection si exemplaires existent
- **Recherche** : Par ISBN, titre, auteur, catégorie, année, mot-clé, disponibilité

#### 4.3.2 Gestion des utilisateurs

- **Inscription** : Génération ID avec préfixe (ETU-, ENS-, ADM-)
- **Validation email** : Regex pour format valide
- **Modification** : Champs génériques et spécifiques par type
- **Suppression** : Blocage si emprunts en cours
- **Recherche** : Par nom, email ou ID (insensible casse)
- **Limites d'emprunt** : 3 (étudiant), 5 (enseignant), 7 (admin)

### 4.3.3 Gestion des emprunts

**Emprunt** : - Vérification disponibilité exemplaire - Contrôle quota utilisateur - Vérification retards bloquants - Durée standard : 14 jours - Mise à jour statistiques automatique

**Retour** : - Transfert vers historique - Gestion file d'attente réservations - Calcul pénalités si retard (100 unités/jour) - Notification premier en attente si applicable

**Renouvellement** : - Extension de 7 jours - Blocage si réservations en attente

### 4.3.4 Système de réservation

- File d'attente FIFO par ISBN
- Notification automatique lors disponibilité
- Écriture dans notifications\_reservations.txt
- Statut exemplaire passe à RESERVE

### 4.3.5 Statistiques et rapports

**Indicateurs générés** : - Total livres et exemplaires - Répartition par statut - Top 5 livres les plus empruntés - Top 5 utilisateurs les plus actifs - Livres jamais empruntés - Total emprunts effectués

**Export CSV** : Format structuré avec horodatage automatique du fichier.

## 4.4 Interfaces utilisateur

### 4.4.1 Interface standard (main.py)

- Menus numérotés classiques
- Saisies textuelles avec validation
- Messages d'erreur explicites
- Confirmations pour actions critiques

### 4.4.2 Interface interactive (main\_interactive.py)

- Navigation au clavier (flèches)
- Menus déroulants avec InquirerPy
- Validation en temps réel
- Icônes et emojis pour meilleure UX
- Fonction effacer\_ecran() pour clarté

**Fonctionnalités communes** : - Système de session (connexion/inscription/invité) - Gestion par rôle (Admin, Enseignant, Étudiant, Invité) - Protection des opérations sensibles - Recherche avancée multicritère

## 5. TESTS ET VALIDATION

### 5.1 Méthodologie de test

Les tests ont été effectués de manière **fonctionnelle** en couvrant tous les cas d'usage normaux et limites.

### 5.2 Scénarios testés

#### 5.2.1 Gestion du catalogue

Ajout livre unique → Succès  
Ajout livre doublon → Message d'erreur approprié  
Ajout exemplaires multiples → IDs séquentiels corrects  
Modification livre → Traçabilité modifications  
Suppression livre avec exemplaires → Blocage  
Suppression livre sans exemplaires → Succès

#### 5.2.2 Gestion des utilisateurs

Inscription étudiant/enseignant/admin → IDs corrects  
Email invalide → Refus avec redemande  
Modification profil → Champs spécifiques préservés  
Suppression avec emprunts actifs → Blocage  
Recherche par mot-clé → Résultats pertinents

#### 5.2.3 Emprunts et retours

Emprunt livre disponible → Statut EMPRUNTE  
Emprunt quota dépassé → Message limite  
Emprunt avec retards existants → Blocage  
Retour simple → Statut DISPONIBLE  
Retour avec réservation → Statut RESERVE + Notification  
Renouvellement sans réservation → Prolongation +7 jours  
Renouvellement avec réservation → Refus

#### 5.2.4 Réservations

Réservation livre indisponible → File d'attente créée  
Retour avec 1 réservation → Notification générée  
Position dans file d'attente → Affichage correct  
Fichier notifications → Format attendu

#### 5.2.5 Persistance

Sauvegarde après ajout → JSON mis à jour  
Chargement au démarrage → Données restaurées

Compteurs globaux → Pas de collision d'IDs  
Historique emprunts → Objets complets reconstruits  
Relations entre objets → Préservées après reload

### 5.2.6 Statistiques

Génération rapport → Tous indicateurs présents  
Export CSV → Fichier créé avec horodatage  
Top 5 livres → Tri décroissant correct  
Livres jamais empruntés → Liste exacte

## 6. DIFFICULTÉS RENCONTRÉES ET SOLUTIONS

### 6.1 Gestion des dépendances circulaires

**Problème** : Les classes Emprunt, Exempleire et Utilisateur se référencent mutuellement.

**Solution** : - Utilisation de références par ID lors de la sérialisation - Reconstruction des objets dans l'ordre de dépendance au chargement - Méthodes from\_dict() acceptant les collections complètes

### 6.2 Persistance de l'historique des emprunts

**Problème initial** : Historique sauvegardé comme liste d'IDs → Perte d'information.

**Solution implémentée** : - Sérialisation complète des objets Emprunt dans to\_dict() - Reconstruction des emprunts passés lors du chargement - Préservation de toutes les données (dates, utilisateur, exemplaire)

### 6.3 Gestion des compteurs globaux

**Problème** : Après chargement, risque de collision d'IDs si compteurs non mis à jour.

**Solution** :

```
if self.__utilisateurs:
 ids = [int(uid.split("-")[1]) for uid in self.__utilisateurs.keys()]
 Utilisateur.compteur = max(ids) + 1
```

Application pour toutes les entités (Exempleire, Emprunt, Reservation).

### 6.4 Encodage des caractères spéciaux

**Problème** : Noms avec accents mal affichés.

**Solution** : encoding="utf-8" systématique sur tous les fichiers (JSON, log, notifications).

## 6.5 Validation des saisies utilisateur

**Problème** : Crashes avec saisies inattendues.

**Solution** : - Validation regex pour emails - Try-except pour conversions numériques - Messages d'erreur explicites - Option 'q' pour annuler partout

## 6.6 Synchronisation interface-métier

**Problème** : Duplication de logique entre les deux interfaces.

**Solution** : Toute la logique dans bibliotheque.py, les interfaces sont de simples wrappers.

---

# 7. AMÉLIORATIONS POSSIBLES

## 7.1 Court terme

**Fonctionnalités** : - Système de pénalités réel avec paiement - Prolongation automatique si pas de réservation - Notifications par email (SMTP) - Import/export catalogue depuis CSV - Code-barres pour ISBN

**Technique** : - Migration vers base de données relationnelle (SQLite/PostgreSQL) - API REST pour accès distant - Tests unitaires avec pytest - Interface graphique (Tkinter/PyQt)

## 7.2 Moyen terme

**Fonctionnalités** : - Gestion des versions de livres (éditions) - Système de recommandation basé sur l'historique - Statistiques avancées (graphiques, tendances) - Gestion des documents numériques (PDF, ePub) - Multi-bibliothèques avec transferts inter-sites

**Technique** : - Architecture microservices - Cache Redis pour performances - Authentification JWT - Logs structurés (JSON) avec ELK Stack - CI/CD avec GitHub Actions

## 7.3 Long terme

**Vision** : - Application mobile (React Native) - Intelligence artificielle pour gestion automatique des stocks - Blockchain pour traçabilité complète - Intégration réseaux sociaux (partage lectures) - Plateforme collaborative multi-établissements

---

# 8. CONCLUSION

## 8.1 Objectifs atteints

Ce projet a permis de développer une **solution complète et fonctionnelle** pour la gestion d'une bibliothèque universitaire. Tous les objectifs du cahier des charges ont été remplis :

**Gestion complète du catalogue** : CRUD livres et exemplaires

**Gestion des utilisateurs** : Trois types avec privilèges différenciés

**Transactions fiables** : Emprunts, retours, renouvellements

**Système de réservation** : File d'attente avec notifications

**Statistiques exploitables** : Rapports détaillés et export CSV

**Persistance robuste** : Sauvegarde automatique et chargement intelligent

**Audit complet** : Journalisation de toutes les opérations

**Interface double** : CLI standard et interactive

## 8.2 Compétences développées

**Programmation Orientée Objet** : - Encapsulation totale avec propriétés Python - Héritage multiple niveaux - Polymorphisme via classe abstraite (ABC) - Patterns de conception (Factory, Template Method, Strategy)

**Gestion de données** : - Sérialisation/désérialisation JSON - Gestion de relations entre objets - Persistance avec intégrité référentielle - Export multi-formats (CSV)

**Développement logiciel** : - Architecture modulaire claire - Gestion d'erreurs robuste - Validation des entrées utilisateur - Documentation complète - Tests fonctionnels

## 8.3 Réflexions personnelles

Ce projet a été une excellente opportunité de mettre en pratique les principes de la POO dans un contexte réel. Les défis rencontrés (dépendances circulaires, persistance, gestion des compteurs) ont permis d'approfondir la compréhension des mécanismes Python.

L'implémentation de deux interfaces (standard et interactive) a démontré l'importance d'une séparation claire entre logique métier et présentation. La phase de test a révélé l'importance de penser aux cas limites dès la conception.

## 8.4 Remerciements

Je tiens à remercier M. Madicke Diop pour l'encadrement de ce projet et les enseignements dispensés tout au long du Master 1 IA. Ce projet a été une expérience formatrice qui a consolidé mes compétences en développement Python.

## 9. BIBLIOGRAPHIE

**Documentation Python** : - Python Software Foundation. (2024). *Python 3.11 Documentation*. <https://docs.python.org/3/> - Python ABC module : <https://docs.python.org/3/library/abc.html>  
- Python Enum : <https://docs.python.org/3/library/enum.html>

**Bibliothèques utilisées** : - InquirerPy Documentation : <https://inquirerpy.readthedocs.io/>  
- JSON en Python : <https://docs.python.org/3/library/json.html>

**Ressources pédagogiques** : - Cours Master 1 IA - DIT (2025-2026)

---

## 10. ANNEXES

### Annexe A : Extraits du fichier JSON

Structure complète du fichier `data_bibliotheque.json` :

```
{
 "catalogue": {
 "978-0123456789": {
 "isbn": "978-0123456789",
 "titre": "Le Petit Prince",
 "auteur": "Antoine de Saint-Exupéry",
 "anne_publication": 1943,
 "categorie": "Jeunesse",
 "statut": "disponible",
 "nb_emprunts_total": 5
 }
 },
 "utilisateurs": {
 "ETU-1": {
 "id_utilisateur": "ETU-1",
 "nom": "Jean Etudiant",
 "email": "jean@dit.sn",
 "type_utilisateur": "ETUDIANT",
 "niveau_etude": "M1",
 "filiere": "IA",
 "liste_emprunt_en_cour": ["EMP-1"],
 "historique_emprunts": [...],
 "nb_emprunts_total": 12
 }
 }
}
```

## Annexe B : Extrait du fichier de logs

[2026-01-06 16:09:14] - UTILISATEUR : Nouvel utilisateur inscrit : Jean Etudiant (ID: ET  
[2026-01-06 16:09:15] - CATALOGUE : Ajout du livre Le Petit Prince (ISBN: 978-0123456789  
[2026-01-06 16:09:15] - EXEMPLAIRE : Nouvel exemplaire créé : EX001-1 pour le livre Le  
[2026-01-06 16:09:15] - EMPRUNT : L'utilisateur ETU-1 a pris l'exemplaire EX001-1  
[2026-01-06 16:09:15] - RESERVATION : L'utilisateur ETU-2 a réservé le livre ISBN: 978-0  
[2026-01-06 16:09:15] - RETOUR : Retour de l'emprunt ID: EMP-1

## Annexe C : Extrait notifications

--- NOTIFICATION DE DISPONIBILITÉ ---

Date : 2026-01-06

Utilisateur : Alice Test (ID: ETU-2)

Livre disponible : Le Petit Prince (ISBN: 978-0123456789)

Action : Merci de passer à la bibliothèque pour récupérer votre exemplaire.

-----

## Annexe D : Exemple de statistiques CSV

Le fichier CSV exporté contient : - Résumé global (total livres, exemplaires, disponibles) - Répartition par statut - Top 5 livres les plus empruntés (titre, ISBN, nb emprunts) - Livres jamais empruntés

**Format horodaté** : statistiques\_bibliotheque\_20260106\_161530.csv

---

## **FIN DU RAPPORT**

*Ce document constitue le rapport complet du projet de gestion de bibliothèque développé dans le cadre de l'examen pratique de Python du Master 1 Intelligence Artificielle au Dakar Institute of Technology.*

**Date de soumission** : 08 Janvier 2026

**Auteur** : SY MAMADOU , SAMBA Japhet Arpaxad , KOYAKPEGON Josué , TONI Lowobien Job Régis